

INSTAGRAM ANALYSIS

PROJECT DESCRIPTION:

In this project I worked as a data analyst for Instagram focusing on analyzing user interactions and engagement using SQL in MySQL Workbench. The objective was to extract meaningful insights from Instagram user data to support decision-making across product, marketing, and development teams. By tracking user behavior and engagement patterns, I provided actionable recommendations that could help improve user experience, guide new feature development and inform targeted marketing campaigns. The insights derived from this analysis aimed to contribute to the growth and future direction of one of the world's most popular social media platforms.



APPROACH :

1.define:The goal of the project was to analyze Instagram database to extract meaningful insights for marketing strategies.user engagement tracking. platform improvement. I followed a structured approach to ensure the analysis was accurate.,efficient, and easy to FIND.

2.collect the data: so for this I frist explored the tables provided: users,photos,likes,comment's ,follows,tags and photo_tags

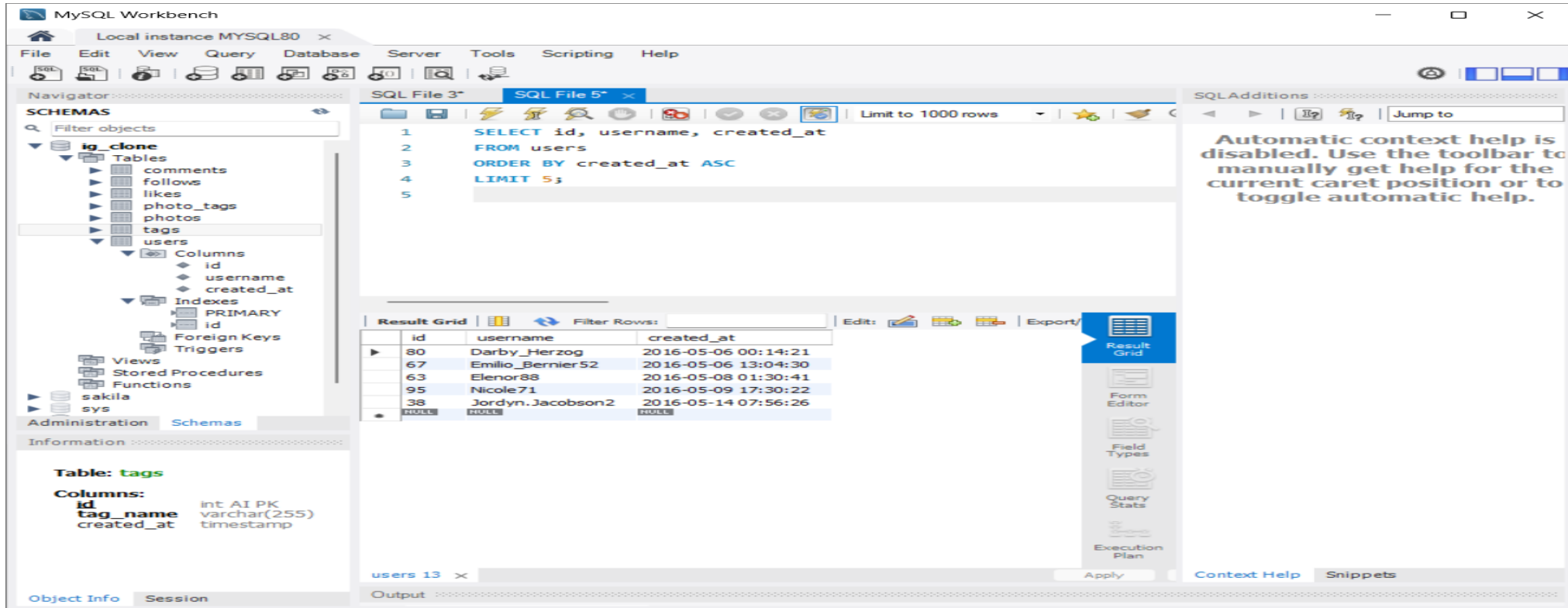
3.clean the data: I identified the relationships between the tables this helped me the joins between needed for the query.ignored the reamaing data

4.analyze it:so I wrote sql quries to do the task provided and analyzed the data and got results

Sql tasks:

A) Marketing Analysis:

1. Loyal User Reward: to find the top 5 loyal users of Instagram to reward them



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ig_clone' expanded, showing tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'users' table is selected, and its columns ('id', 'username', 'created_at') and indexes are visible. The main editor shows a SQL query: `SELECT id, username, created_at FROM users ORDER BY created_at ASC LIMIT 5;`. The 'Result Grid' at the bottom displays the top 5 users sorted by creation date. The right sidebar contains a 'SQLAdditions' panel with a message about context help.

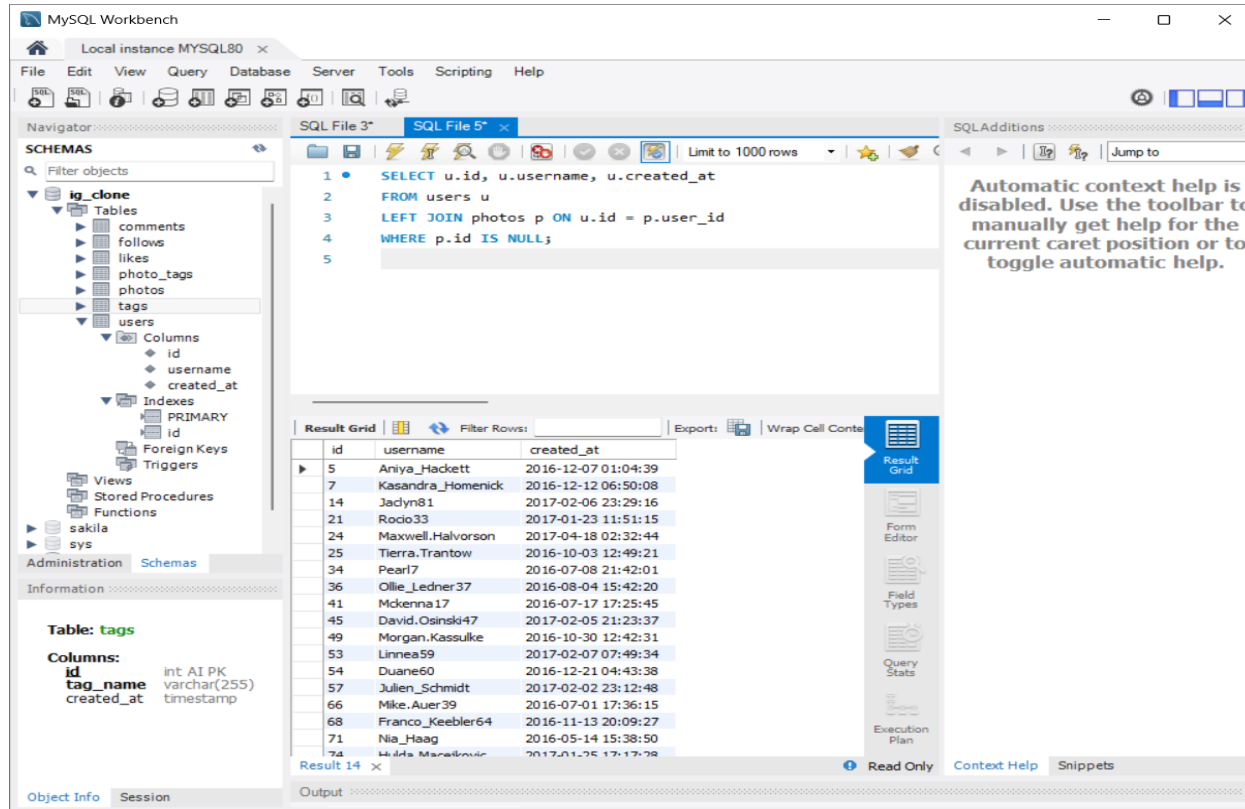
id	username	created_at
80	Darby_Herzog	2016-05-06 00:14:21
67	Emilio_Bernier52	2016-05-06 13:04:30
63	Elenor88	2016-05-08 01:30:41
95	Nicole71	2016-05-09 17:30:22
38	Jordyn.Jacobson2	2016-05-14 07:56:26

Sorted the users table by created_at in ascending order.

I set the limits up to 5.

And got the top 5 loyal users

2. Inactive User Engagement:



I used the left joint between users and photos on userids.
Filtered the rows where photos.id is null.and got the insights

id	username	created_at	5	Aniya_Hackett
		2016-12-07 01:04:39	7	Kassandra_Homenick
		2016-12-12 06:50:08	14	Jaclyn81
23:29:16	21	Rocio33	2017-01-23 11:51:15	24
	Maxwell.Halvorson	2017-04-18 02:32:44	25	
	Tierra.Trantow	2016-10-03 12:49:21	34	
	Pearl7	2016-07-08 21:42:01	36	
	Ollie_Ledner37	2016-08-04 15:42:20	41	
	Mckenna17	2016-07-17 17:25:45	45	
	David.Osinski47	2017-02-05 21:23:37	49	
	Morgan.Kassulke	2016-10-30 12:42:31	53	
	Linnea59	2017-02-07 07:49:34	54	Duane60
		2016-12-21 04:43:38	57	Julien_Schmidt
		2017-02-02 23:12:48	66	Mike.Auer39
		2016-07-01 17:36:15	68	Franco_Keebler64
		2016-11-13 20:09:27	71	Nia_Haag
15:38:50	74	Hulda.Macejkovic	2017-01-25 17:17:28	
	75	Leslie67	2016-09-21 05:14:01	76
	Janelle.Nikolaus81	2016-07-21 09:26:09	80	
	Darby_Herzog	2016-05-06 00:14:21	81	
	Esther.Zulauf61	2017-01-14 17:02:34	83	
	Bartholome.Bernhard	2016-11-06 02:31:23	89	
	Jessyca_West	2016-09-14 23:47:05	90	
	Esmeralda.Mraz57	2017-03-03 11:52:27	91	
	Bethany20	2016-06-03 23:31:53		

3. Contest Winner Declaration:

Goal is to find the photo with highest like count and who posted it.

The screenshot shows the MySQL Workbench interface. On the left, the 'Navigator' pane displays the database schema 'ig_clone' with tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'Information' pane shows details for the 'tags' table. The central 'SQL Editor' pane contains the following query:

```
1 SELECT p.id AS photo_id, p.image_url, COUNT(l.user_id) AS like_count
2 FROM photos p
3 JOIN likes l ON p.id = l.photo_id
4 GROUP BY p.id, p.image_url
5 ORDER BY like_count DESC
6 LIMIT 1;
```

Below the query, the 'Result Grid' shows the output:

photo_id	image_url	like_count
145	https://jarret.name	48

On the right, the 'SQLAdditions' pane displays a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

- I used the join approach between photos and likes per photo
- used the group by photo id and order by count(likes) descending.
- Limit set to the 1

4.Hashtag Research:

To find the most used hashtags

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a database named 'ig_clone' with tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'tags' table is selected, and its structure is shown in the 'Table: tags' pane: columns are 'id' (int AI PK), 'tag_name' (varchar(255)), and 'created_at' (timestamp). The main editor shows a SQL query in 'SQL File 5*':

```
1 SELECT t.tag_name, COUNT(*) AS usage_count
2 FROM tags t
3 JOIN photo_tags pt ON t.id = pt.tag_id
4 GROUP BY t.id, t.tag_name
5 ORDER BY usage_count DESC
6 LIMIT 5;
```

The 'Result Grid' pane shows the results of the query:

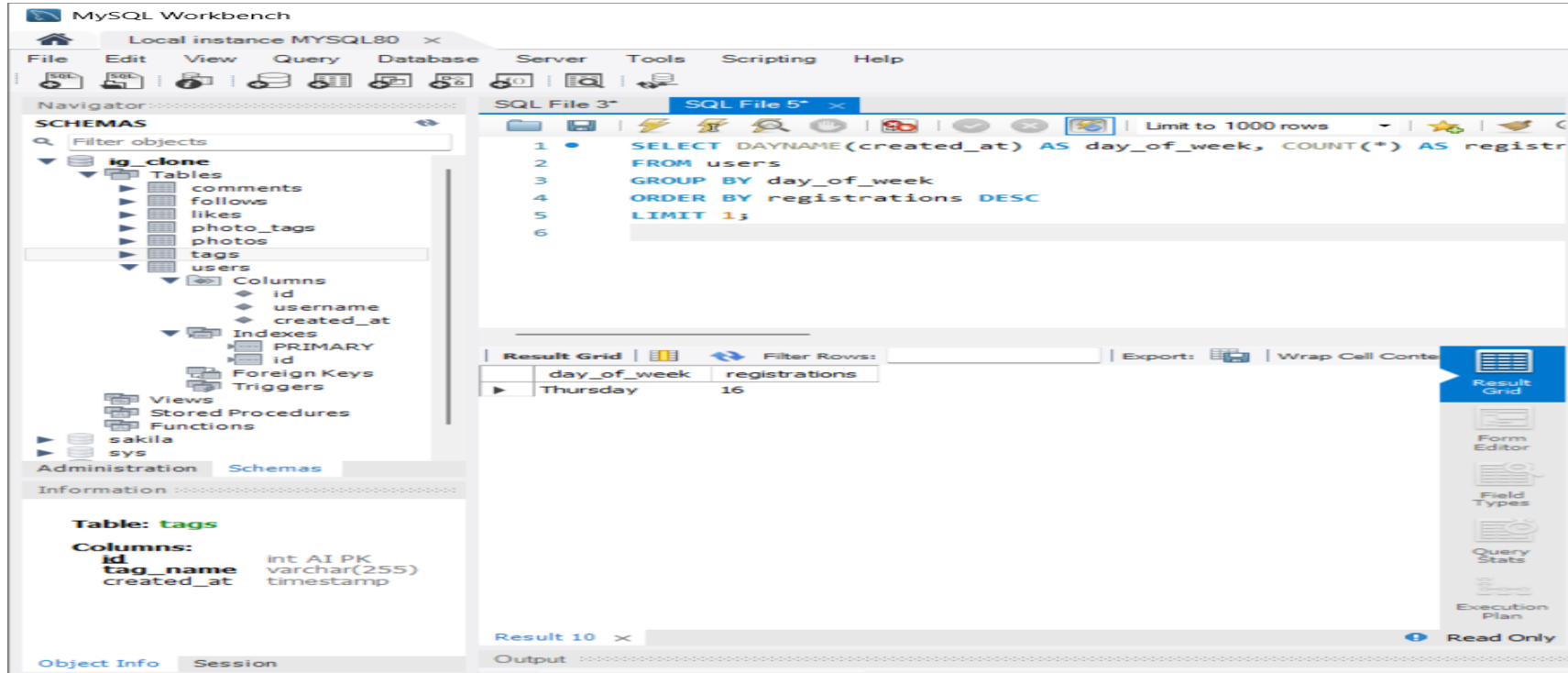
tag_name	usage_count
smile	59
beach	42
party	39
fun	38
concert	24

On the right, the 'SQLAdditions' pane displays a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

- Joined the tags,photo tags,photos to count the usage
- Group by tag_name and order by count descending
- Limits set to top 5

5. Ad Campaign Launch:

To find the day when most user resigter.



The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'users' table is selected, showing its columns: 'id' (int AI PK), 'username' (varchar(255)), and 'created_at' (timestamp). Below this, the 'Table: tags' structure is also visible.

The central SQL editor shows the following query:

```
1 SELECT DAYNAME(created_at) AS day_of_week, COUNT(*) AS registr
2 FROM users
3 GROUP BY day_of_week
4 ORDER BY registrations DESC
5 LIMIT 1;
6
```

The 'Result Grid' at the bottom displays the query results:

day_of_week	registrations
Thursday	16

The right sidebar contains various tool icons, including 'Result Grid', 'Form Editor', 'Field Types', 'Query Stats', and 'Execution Plan'.

- I extract the day name from created_at from users table
- Group by the day and count users.
- Picked the day with maximum count .

B) Investor Metrics:

1. User Engagement: to find average posts by user

- I calculate total posts: count(*) from photos
- calculate total users from users
- Divided posts by users for average posts per users.

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'ig_clone' database selected, showing tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'users' table is expanded, showing columns 'id', 'username', and 'created_at'. The main editor window shows a SQL query in 'SQL File 5*':

```
1 SELECT
2     COUNT(p.id) / COUNT(DISTINCT u.id) AS avg_posts_per_user
3 FROM users u
4 LEFT JOIN photos p ON u.id = p.user_id;
5
6
```

The query is executed, and the 'Result Grid' shows the following result:

avg_posts_per_user
2.5700

The bottom status bar indicates 'Result 15' and 'Read Only'.

Information panel (Table: tags):

Columns:	
id	int AI PK
tag_name	varchar(255)
created_at	timestamp

2. Bots & Fake Accounts:

to find the accounts that liked the every single photo to find the fake accounts.

- First, get the total photos from the photos table
- Then count likes from likes table
- Count = total photos.

The screenshot shows the MySQL Workbench interface. On the left, the 'Navigator' pane displays the 'ig_clone' database schema, including tables like 'comments', 'follows', 'likes', 'photo_tags', 'photos', 'tags', and 'users'. The 'Users' table is expanded, showing columns 'id', 'username', and 'created_at'. Below the schema, the 'Table: tags' is detailed with columns 'id', 'tag_name', and 'created_at'. The main editor shows a SQL query in 'SQL File 5' that counts the number of likes for each user, comparing it to the total number of photos. The query is as follows:

```
1 SELECT l.user_id, u.username, COUNT(l.photo_id) AS liked_photos_count
2 FROM likes l
3 JOIN users u ON l.user_id = u.id
4 GROUP BY l.user_id, u.username
5 HAVING liked_photos_count = (SELECT COUNT(*) FROM photos);
6
```

The 'Result Grid' at the bottom displays the results of the query, showing a list of users and their corresponding 'liked_photos_count'. The results are as follows:

user_id	username	liked_photos_count
5	Aniya_Hackett	257
14	Jadyn81	257
21	Rocio33	257
24	Maxwell.Halvorson	257
36	Ollie_Ledner37	257
41	Mckenna17	257
54	Duane60	257
57	Julien_Schmidt	257
66	Mike.Auer39	257
71	Nia_Haag	257
75	Leslie67	257
76	Janelle.Nikolaus81	257
91	Bethany20	257

The right side of the interface shows the 'SQLAdditions' pane with a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

Tech -stack used

1. For this project, I worked mainly with) **MySQL (Version 8.0)** as the database system. It's a reliable and widely-used tool for handling relational data, and I chose it because it's fast, efficient, and has great community support, which makes troubleshooting easier.
2. I used **MySQL Workbench (Version 8.0 CE)** to write and run my SQL queries. It provides a clean, visual interface where I could easily manage the database, check table structures, and debug my queries without having to work entirely in the command line.
3. The main language I used was **SQL (Structured Query Language)**. This is what I relied on to retrieve, filter, and combine data from different tables. SQL made it possible to run detailed analyses, find patterns, and answer the business questions for the project.

Insights:

While analyzing the Instagram dataset, I noticed some interesting trends and patterns.

- The five oldest users have been around for a long time, but not all of them are still posting regularly.
- Many users have never posted a single photo, which could be a good opportunity for re-engagement campaigns.
- The contest winner stood out with an exceptionally high number of likes on one photo, showing how certain posts can go viral.
- Popular hashtags tend to appear across multiple posts, making them valuable for boosting marketing reach.
- User sign-ups peak on certain days of the week, which could help in deciding the best time to launch ad campaigns.
- On average, users are posting fewer photos than expected, pointing to a drop in overall engagement.
- On average, users are posting fewer photos than expected, pointing to a drop in overall engagement.

Results

Through this project, I was able to uncover valuable insights about user behavior and engagement on Instagram. The analysis helped identify highly active users, inactive accounts, trending hashtags, and even suspicious activity that could indicate bots. These findings can directly support marketing strategies, improve user engagement campaigns, and guide ad scheduling for better reach. On a personal level, the project strengthened my SQL skills, improved my ability to work with real-world datasets, and gave me hands-on experience in turning raw data into actionable business insights.

